



ขอให้ถือผลประโยชน์ส่วนตัวเป็นที่สอง
ประโยชน์ของเพื่อนมนุษย์เป็นกิจที่หนึ่ง
ลาภทรัพย์สินและเกียรติยศจะตกมาแก่ท่านเอง
ถ้าท่านทรงธรรมะแห่งอาชีพไว้ให้บริสุทธิ์

ในพระ



344-111

Greedy Algorithm อัลกอริทึมเชิงละโมภ



Greedy Algorithm

- ขั้นตอนวิธีการแก้ปัญหาที่คิดแบบง่ายๆ และตรงไปตรงมา
- วิธีการนี้จะเลือกทางเลือกที่ให้ผลตอบแทนคุ้มค่าที่สุดตามข้อมูลที่มีอยู่ในขณะนั้น
- Greedy Algorithm ใช้กับปัญหาเหมาะสมที่สุด Optimization problem เพราะว่า เราต้องการการตัดสินใจ ว่าทางเลือกในปัจจุบันมีค่าตอบแทนมากที่สุด หรือน้อยที่สุด หรือไม่



ตัวอย่างปัญหาที่ใช้วิธีคิดแบบ Greedy

- Coin Changing
 - Fractional Knapsack
 - Activity Selection
 - Huffman Code → tree
 - Prim's
 - Kruskal's
 - Dijkstra's
- graph



ปัญหาการทอนเหรียญ (Coin Changing)

- ในร้านขายของเล็ก ๆ แห่งหนึ่งในประเทศอเมริกา คนขายมักจำเป็นต้องทอนเงินเป็นเหรียญให้กับลูกค้า โดย กำหนดว่ามีเหรียญ 5 ประเภท
 - เหรียญ dollar (มีค่า 1)
 - เหรียญ quarter (มีค่า 0.25)
 - เหรียญ dime (มีค่า 0.10)
 - เหรียญ nickel (มีค่า 0.05)
 - เหรียญ penny (มีค่า 0.01)
- **ปัญหาคือ** ทอนเงินอย่างไรให้ใช้เหรียญน้อยที่สุด

ex. $C = \{10, 5, 2, 1\}$ out put

2¢ $\begin{cases} 10 \times 2 \\ 5 \times 1 \\ 2 \times 1 \\ 1 \times 1 \end{cases}$

step

1. $\frac{2¢}{10} = 2$, $2¢ \% 10 = 2$

2. $\frac{2}{5} = 1$, $2 \% 5 = 2$

3. $\frac{2}{2} = 1$, $2 \% 2 = 0$

4. $\frac{1}{1} = 1$



ปัญหาการทอนเหรียญ

- เช่น หากต้องการทอนเงิน \$3.69

$\$3.69 = 3 \text{ เหรียญ dollar} + 2 \text{ เหรียญ quarter} + 1 \text{ เหรียญ dime} + 1 \text{ เหรียญ nickel} + 4 \text{ เหรียญ penny}$

จะได้ว่าต้องทอนเหรียญ

- dollar 3 เหรียญ
- quarter 2 เหรียญ
- dime 1 เหรียญ
- nickel 1 เหรียญ
- penny 4 เหรียญ

จึงจะใช้จำนวนเหรียญน้อยที่สุด



Algorithm: CHANGE-Greedy

- **ข้อมูลเข้า** y เป็นจำนวนเต็มบวก หรือ 0
- **ข้อมูลออก** $(x_1, x_2, x_3, x_4, x_5)$ โดยที่ $x_1 - x_5$ เป็นจำนวนเต็มบวกที่มากกว่าหรือเท่ากับศูนย์
 - x_1 แทนจำนวนเหรียญ dollar
 - x_2 แทนจำนวนเหรียญ quarter
 - x_3 แทนจำนวนเหรียญ dime
 - x_4 แทนจำนวนเหรียญ nickel
 - x_5 แทนจำนวนเหรียญ penny



Algorithm: CHANGE-Greedy

```
1.  c = {100 , 25 , 10 , 5 , 1};
2.  x = {0 , 0 , 0 , 0 , 0 } ;
3.  sum = y ;
4.  i = 1;
5.  While sum != 0
    1. { if (sum > C[i] )
        1.  {
        2.    x[i] = sum div c[i]
        3.    sum = sum - (c[i] * x[i])
        4.    }
    2. } //end while
```

แนวคิด
เลือกเหรียญที่มีค่า
มากที่สุดก่อน

$C = \{1, 3, 4\}$

money = 6

output Greedy

$\frac{6}{4} = 1$, $6 \% 4 = 2$

$\frac{2}{1} = 2$, $2 \% 1 = 0$

Not Best†

sol by Dynamic prog



ปัญหาถุงเป้ (Fractional Knapsack)

- **กำหนดให้:** มีสิ่งของ n ประเภท ซึ่งแต่ละประเภท (i) กำหนดให้มีจำนวน x_i ชิ้น มีค่าความสำคัญ b_i และมีน้ำหนัก w_i
- **ปัญหา:** ต้องการหาจำนวนสิ่งของแต่ละประเภทที่บรรจุลงในเป้ที่รับน้ำหนักได้ไม่เกิน W กิโลกรัม
- **แนวคิด:** ให้เลือกหยิบสิ่งของที่ละชิ้นที่มีค่าดัชนี ($v_i = b_i/w_i$) สูงสุดและทำให้น้ำหนักรวมไม่เกิน W ก่อน



Fractional Knapsack

step 1: คำนวณ $V_i = \frac{b_i}{w_i}$

step 2: sort item by V

step 3: Select ตาม V_{max}
ก็คือไม่ใช้ทุกขนาด

ตัวอย่างเช่น มีของ 4 ประเภทคือ

1. หนังสือ 4 เล่ม มี $b_1=10$ และ $w_1=0.6$ ($v_1=16.7$)
2. ขนอม 2 กล่อง มี $b_2=7$ และ $w_2=0.4$ ($v_2=17.5$)
3. น้ำ 2 ขวด มี $b_3=5$ และ $w_3=0.5$ ($v_3=10$)
4. ซีดีเพลง 8 แผ่น มี $b_4=3$ และ $w_4=0.2$ ($v_4=15$)

② Sort by V

1. ขนอม
2. หนังสือ
3. ซีดี
4. ขวดน้ำ

กำหนดให้ เป้ารับน้ำหนักได้ไม่เกิน 4 กิโลกรัม จะได้ลำดับการเลือกดังนี้

1. ขนอม 2 กล่อง (น้ำหนักรวม $2 \times 0.4 = 0.8$)
2. หนังสือ 4 เล่ม (น้ำหนักรวม $4 \times 0.6 = 2.4$)
3. ซีดีเพลง 4 แผ่น (น้ำหนักรวม $4 \times 0.2 = 0.8$)

③

$$0.8 + 2.4 + 0.8 = 4.0$$

ได้ค่าความสำคัญสูงสุดและน้ำหนักไม่เกิน 4 กิโลกรัม



Algorithm: Fractional Knapsack

- จงเขียนขั้นตอนวิธีการ โดยแสดงด้วย Pseudo-Code เพื่อแสดงขั้นตอนการทำงานของปัญหา Fractional Knapsack
- Input: จำนวนสิ่งของ (n) ความสำคัญ (b_i) และน้ำหนัก (w_i) ของสิ่งของแต่ละชิ้น
- Output: สิ่งของที่ถูกเลือกและจำนวนชิ้นเรียงตามลำดับ และน้ำหนักรวมมากที่สุด



ปัญหาการเลือกกิจกรรม (Activity Selection)

- กำหนดให้ $A = \{a_1, a_2, \dots, a_n\}$ คือเซตของการประชุมต่างๆ ที่มีผู้ทำเรื่องขอใช้ห้องประชุมห้องหนึ่ง
- การประชุมที่ a_i มีกำหนดการใช้ห้องประชุมตั้งแต่เวลา s_i ถึง f_i ($s_i < f_i$)
- ซึ่งอาจเป็นไปได้ว่าอาจมีการขอใช้ห้องประชุมห้องนี้ในเวลาที่เหลื่อมกัน
- **ปัญหาคือ** จะอนุมัติให้การประชุมใดบ้าง มีสิทธิ์ใช้ห้องประชุมห้องนี้ เพื่อให้ได้การประชุมมากที่สุด และไม่มีการประชุมที่ใช้เวลาเหลื่อมกัน

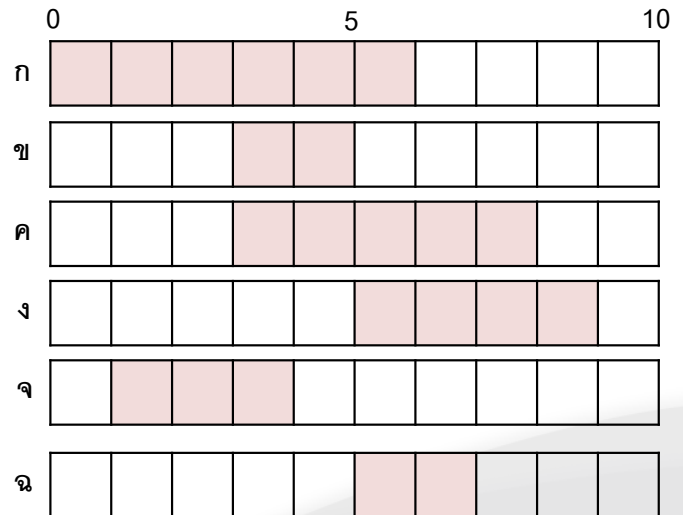


ปัญหาการเลือกกิจกรรม

- ตัวอย่างเช่น มีกิจกรรมอยู่ 6 กิจกรรม เวลาเริ่มต้นและสิ้นสุดแต่ละ

กิจกรรมแสดงดังตาราง

a_i	ก	ข	ค	ง	จ	ฉ
s_i	0	3	3	5	1	5
f_i	6	5	8	9	4	7

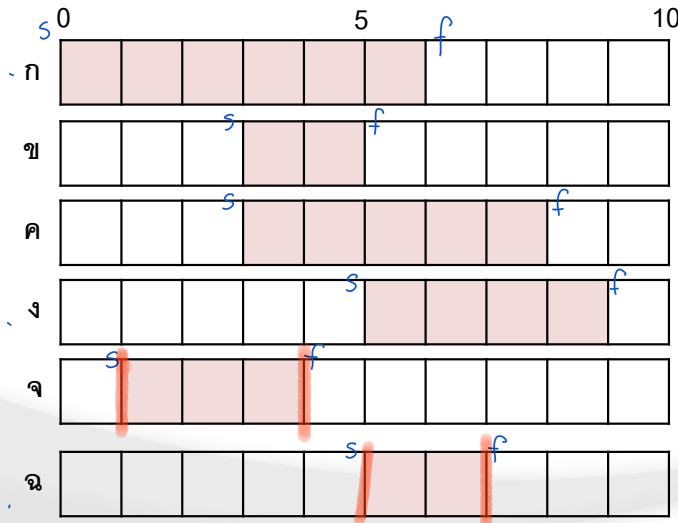




ปัญหาการเลือกกิจกรรม

- ① sort a ตาม f จากน้อยไปมาก
- ② เลือกพิจารณาตามลำดับแสงเงา

- แนวคิดการแก้ปัญหา เลือกกิจกรรมซึ่งมีเวลาสิ้นสุดของการใช้ทรัพยากร (f) น้อยที่สุด ที่ไม่เหลื่อมกับกิจกรรมที่ได้เลือกไว้แล้ว

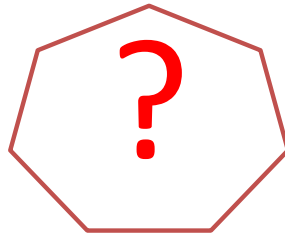


กิจกรรมที่ถูกเลือกอันดับที่ 1 คือ จ
กิจกรรมที่ถูกเลือกอันดับที่ 2 คือ ฉ
ผลเฉลยคือ {จ, ฉ}

จ → ข → ก → ฉ → ค → ง
× × ✓ × ×



Algorithm: GreedyActivitySelect(a[1..n])



แนวคิด

1. เรียงลำดับกิจกรรมตามเวลาสิ้นสุดของกิจกรรมจากน้อยไปมาก (ใช้เวลา $O(n \log n)$)
2. วนลูปเพื่อเลือกกิจกรรม (ใช้เวลา $\Theta(n)$)

ใช้เวลารวมเป็น $O(n \log n) + \Theta(n) = O(n \log n)$



Huffman code

- เป็นขั้นตอนวิธีการลดขนาดของข้อมูลที่มีจำนวนมากโดยที่สามารถนำข้อมูลแปลงกลับมาในสภาพที่เหมือนเดิมได้
- หลักการคือการใช้ตารางความถี่การปรากฏของแต่ละตัวอักษร
- เป็นวิธีการแทนที่ตัวหนังสือ ที่น้อยที่สุดโดยใช้ Binary String
- โดยทั่วไปสามารถที่จะลดขนาดของข้อมูลได้ 20% ถึง 90%



Huffman code

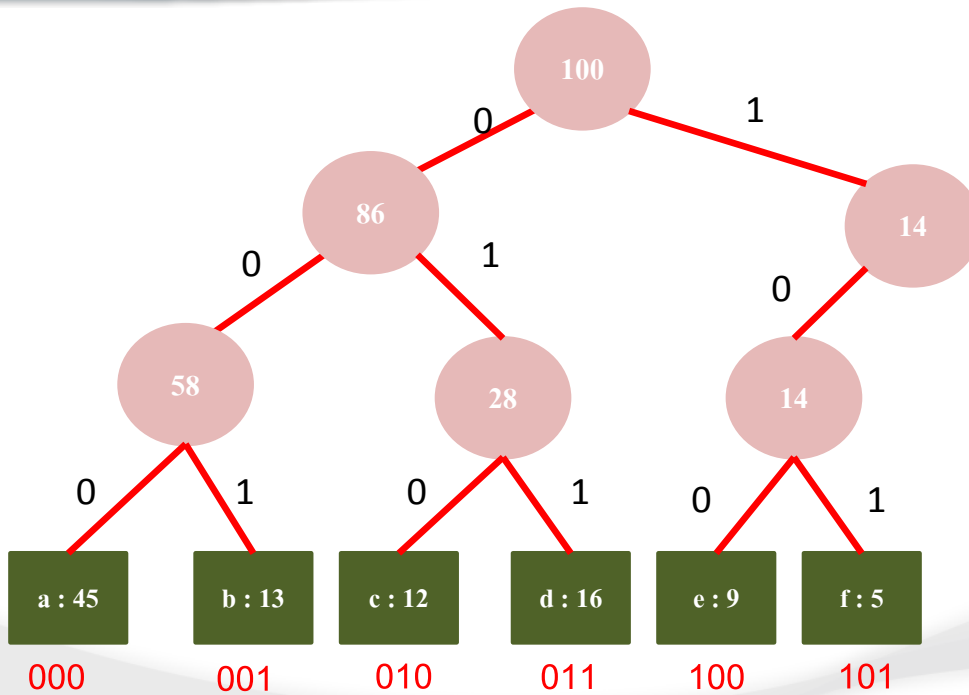
$$\begin{aligned}2^1 &= 0 \quad 1 \\2^2 &= 00 \quad 01 \quad 10 \quad 11 \\2^3 &= 000 \quad 001 \dots\end{aligned}$$

- ตัวอย่างชุดข้อมูล

	a	b	c	d	e	f
Frequency	45	13	12	16	9	5
Fix-Length	000	001	010	011	100	101
Variable-length	0	101	100	111	1101	1100



Fixed - Length

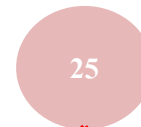




① สร้าง Huffman tree ② Huffcode

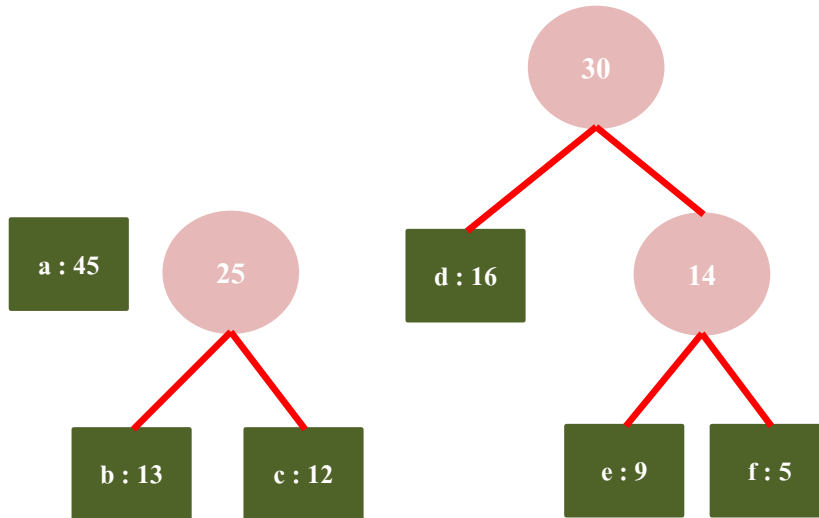
Variable – Length : Huffman code Tree

step 1: sort อักษรตามถี่ จากมากไปน้อย



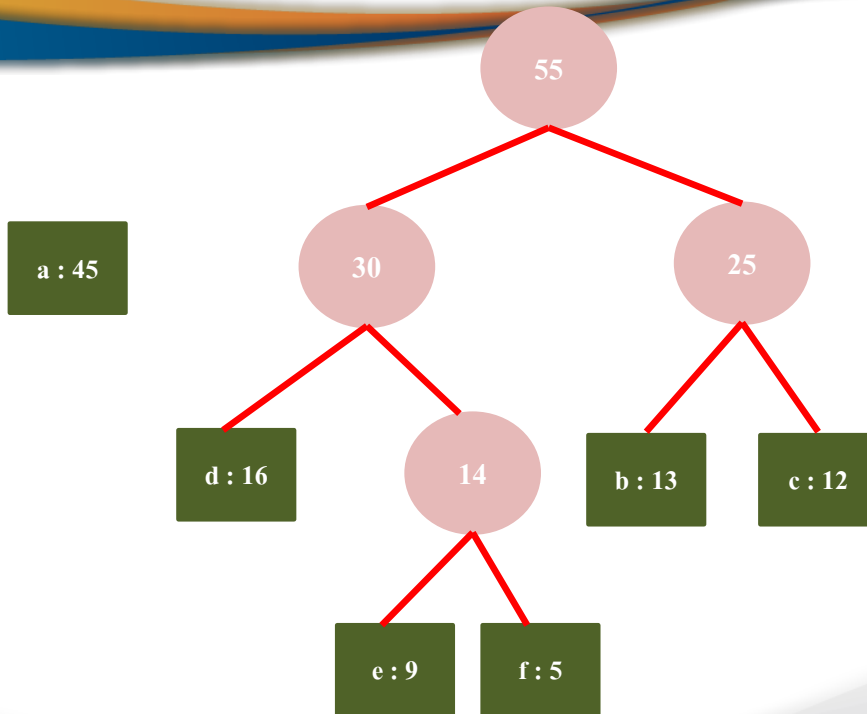


Variable – Length : Huffman code Tree





Variable – Length : Huffman code Tree

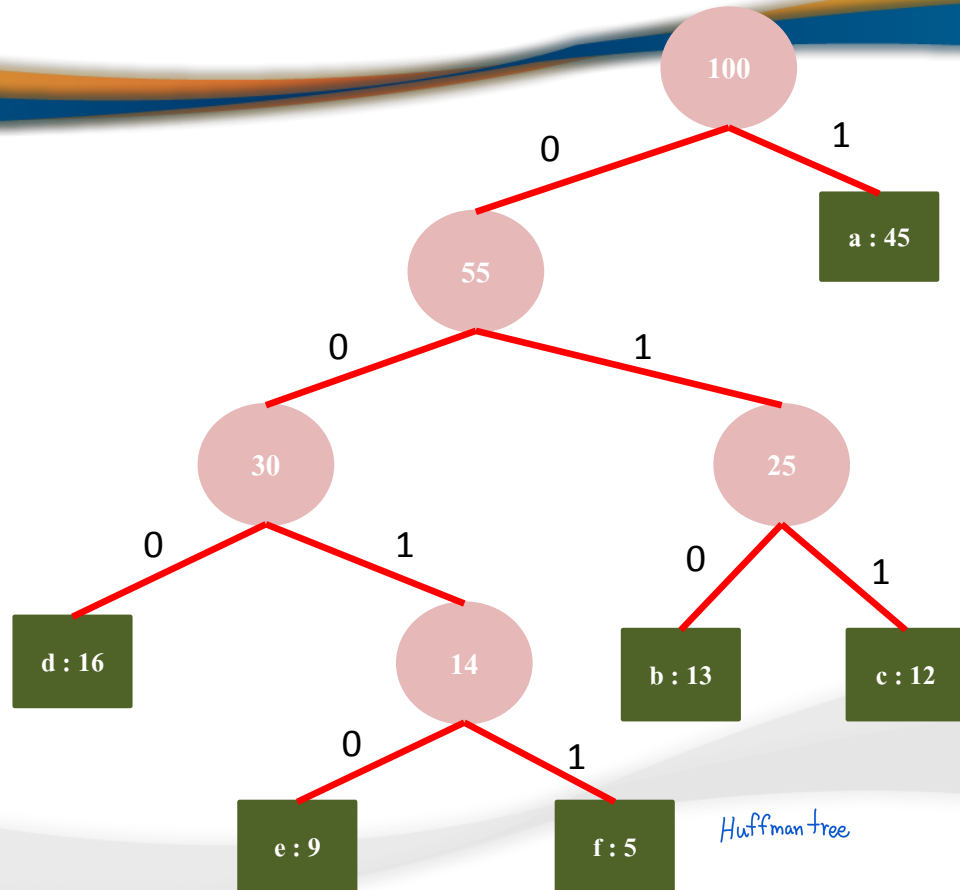




Variable – Length : Huffman code Tree

Huffman code

- a แทนด้วยรหัส 1
- b แทนด้วยรหัส 010
- c แทนด้วยรหัส 011
- d แทนด้วยรหัส 000
- e แทนด้วยรหัส 0010
- f แทนด้วยรหัส 0011



Huffman tree



Huffman Tree

Algorithm:

Input: Array W of character probabilities

Output: The Huffman tree.

- 1: **for** $i \leftarrow 0$ to $n - 1$ **do**
- 2: $T \leftarrow$ node labeled with character i
- 3: $T.\text{weight} \leftarrow W[i]$; add T to set S
- 4: **for** $i \leftarrow 0$ to $n - 2$ **do**
- 5: $L, R \leftarrow$ remove two min. weight trees from S
- 6: $T \leftarrow$ node with children L and R
- 7: $T.\text{weight} \leftarrow L.\text{weight} + R.\text{weight}$
- 8: add T to set S
- 9: return T



Huffman Tree

Analysis:

- Let n be the number of words.
- The time complexity is $O(n \log n)$ when using heap to store the weight.



ปัญหาอื่นๆ

- Prim's Algorithm -> <https://www.youtube.com/watch?v=Pn874kEc3IA>
- Kruskal's Algorithm -> <https://www.youtube.com/watch?v=5XkK88VEILk>
- Dijkstra's Algorithm -> <https://www.youtube.com/watch?v=1057z9XTfcs>
- <https://www.youtube.com/watch?v=WOCV2UcxNrI>